

Project Report

AI Career Companion Agent for Internship Matching and Interview Preparation -

Technical documentation, Evaluation and Final Report

Submitted by

Afza Fathima

Computer Science and Engineering

Kalpataru Institute of Technology, Tiptur

AI Intern @Infosys Springboard

Contents

1. Introduction.....	4
1.1 What was built.....	4
1.2 Organisation of this report	4
2. Problem Statement	5
3. Objectives	6
3.1 Main objective	6
3.2 Specific objectives	6
4. System Requirements.....	7
4.1 Functional requirements	7
4.2 Non-functional requirements.....	7
4.3 Software and hardware requirements.....	8
5. System Architecture	9
5.1 Components	9
5.2 Data flow	10
5.3 Key design decisions.....	10
6. Dataset and Knowledge Base	11
6.1 Overview.....	11
6.2 Job categories.....	11
6.3 Preprocessing	11
7. RAG Pipeline	12
7.1 Chunking.....	12
7.2 Embedding.....	12
7.3 Index and search.....	12
7.4 Retrieval quality.....	12
8. Multi-Agent Architecture	14
8.1 Coordination pattern.....	14
9. Agent Design and Implementation.....	16
9.1 Job-Resume Matching Agent.....	16
9.2 Skill Gap Analysis Agent.....	16
9.3 Resume & Cover Letter Customization Agent.....	16
9.4 Interview Preparation Agent	16
9.5 Conversational Career Assistant.....	17
9.6 Resume processing and LLM integration	17
10. Application Tracking Module.....	18
11. Testing and Evaluation	19
11.1 Strategy	19
11.2 Retrieval evaluation (precision@5)	19

11.3 Score consistency	19
11.4 What was not evaluated	20
12. Results and Analysis	21
12.1 Worked example	21
12.2 Analysis	21
13. Limitations	22
14. Future Scope	23
15. Conclusion	24
References	24
Appendix A. Final System Demonstration	Error! Bookmark not defined.
A.1 Before you start	Error! Bookmark not defined.
A.2 Walkthrough	Error! Bookmark not defined.
A.3 Verified output of the non-LLM steps	Error! Bookmark not defined.
A.4 What to check on the live run	Error! Bookmark not defined.
A.5 Points to avoid or explain during the demo	Error! Bookmark not defined.

1. Introduction

Finding a suitable internship is one of the first real career decisions a student makes, and it is a poorly supported one. Postings are scattered across portals, written in inconsistent formats, and rarely say how well a particular student fits. Students then have to repeat the same work for every application: tailor a resume, write a cover letter, work out which skills they lack, prepare for interviews, and keep track of where each application stands.

AI Career Companion is a web application that supports this whole journey in one place. A student uploads a resume; the system extracts a structured profile, retrieves relevant internships from a knowledge base using Retrieval-Augmented Generation (RAG), scores the student against them with an explainable formula, identifies skill gaps, tailors resume and cover-letter content, generates interview practice, answers follow-up questions in a chat assistant, and tracks every application through its lifecycle.

The system is built as a cooperating set of AI agents on top of a shared student profile and a shared internship knowledge base. A guiding choice runs through the design: anything that must be exact (scores, gap categories, dashboard counts, reminders) is computed by deterministic, tested code, and the language model is used only where language is genuinely needed, with its output validated before it is shown.

1.1 What was built

- A React frontend with eleven pages and a FastAPI backend with ten routers and 28 endpoints.
- A resume-processing pipeline: PDF/DOCX → text → LLM structured extraction → relational student profile.
- An internship knowledge base of 200 postings (400 chunks) and a local vector index with semantic search.
- Five agents: Job-Resume Matching, Skill Gap Analysis, Resume & Cover Letter Customization, Interview Preparation and the Conversational Career Assistant.
- An application-tracking module with ten status stages, six dashboard metrics and four reminder types.
- An automated test suite (20 tests), a retrieval evaluation harness, and an end-to-end demonstration script.

1.2 Organisation of this report

Sections 2–4 state the problem, objectives and requirements. Sections 5–10 describe the architecture, knowledge base, RAG pipeline, agents and tracking module. Sections 11–12 report testing and results, Sections 13–14 limitations and future work, and Section 15 concludes. Appendix A documents the final demonstration.

2. Problem Statement

Students searching for internships face several connected problems:

1. **Discovery is keyword-driven.** Portals match on titles and exact words, so a student with React and CSS skills may never see a “Web Design” role, and a query like “react” can match unrelated text.
2. **Fit is opaque.** A percentage with no explanation does not tell a student what to fix. Students need to know which required skills they have, which they lack, and whether experience or education requirements are a barrier.
3. **Skill gaps are not actionable.** Knowing a skill is missing is different from knowing how important it is for this role and what to do about it.
4. **Applications are generic.** Tailoring a resume and cover letter to each posting is slow, and generative tools can invent skills or achievements the student does not have, which is harmful in a job application.
5. **Interview preparation is unfocused.** Generic question lists ignore the student’s own resume, projects and weak spots.
6. **Tracking is manual.** Deadlines, interview dates and follow-ups are kept in notes or spreadsheets and are easily missed.

Problem to solve

Build a single system that turns a resume into relevant, explained internship recommendations and supports the student through tailoring, preparation and tracking, while keeping generated content grounded in what the student has actually done.

3. Objectives

3.1 Main objective

Design and implement an AI-powered career companion that supports the complete internship workflow for students using RAG and a multi-agent architecture.

3.2 Specific objectives

#	Objective	Delivered by
O1	Create and manage student profiles; extract skills, education, experience, projects and certifications from an uploaded resume.	students and resumes routers, text_extraction.py, llm.py
O2	Build a reproducible internship knowledge base from a public dataset.	data_pipeline/ (200 postings)
O3	Implement a RAG pipeline: chunking, embedding, indexing and semantic search.	rag/
O4	Recommend and score internships with an explainable breakdown.	Job-Resume Matching Agent
O5	Classify skill, experience and education gaps and recommend how to close them.	Skill Gap Analysis Agent
O6	Generate tailored resume content and cover letters without inventing skills.	Customization Agent with validation
O7	Prepare the student for interviews across five question categories and evaluate practice answers.	Interview Preparation Agent
O8	Provide a conversational interface that retains context across turns and answers from real agent output.	Conversational Career Assistant
O9	Track applications, deadlines and interviews with a dashboard and reminders.	Application Tracking Module
O10	Test and evaluate the system and document it fully.	tests/, evaluation/, this report and the technical documentation

4. System Requirements

4.1 Functional requirements

ID	Requirement
FR1	The system shall let a student create a profile with name, email (unique) and target role.
FR2	The system shall accept resume uploads in PDF or DOCX format up to 5 MB and reject other types and larger files.
FR3	The system shall extract skills, education, experience, projects and certifications from a resume into structured records.
FR4	The system shall search the internship knowledge base semantically from free text.
FR5	The system shall recommend internships for a student with an overall score and separate skill, education and experience scores.
FR6	The system shall list matched required skills, missing required skills and matched preferred skills for each recommendation, with a short explanation.
FR7	The system shall classify gaps for a selected internship as critical, partially demonstrated or preferred, and report experience and education gaps.
FR8	The system shall generate a tailored resume and a cover letter for a selected internship, accept feedback for regeneration, and flag skills not present in the student's profile.
FR9	The system shall generate interview questions in five categories, identify revision topics, and evaluate practice answers.
FR10	The system shall provide a chat assistant that recommends, explains matches and gaps, compares internships, generates materials and prepares interviews, remembering the internship under discussion.
FR11	The system shall track applications through ten defined statuses, with deadlines, interview dates, notes and the resume/cover letter used.
FR12	The system shall show dashboard metrics and reminders, and support filtering by company, title, status and deadline.

4.2 Non-functional requirements

Quality	Requirement and how it is met
Explainability	Scores come from a documented formula; the LLM explains computed results and is told not to recompute them.
Groundedness	Generated documents use the student's profile as the sole source of truth and are checked for unsupported skills; the assistant answers only from gathered agent output.
Reliability	Every LLM call has a defined fallback when the model returns invalid JSON; invalid statuses and intents are rejected or normalised.
Performance	Retrieval and scoring run locally with no model download (about 1.2 s cold for a 15-candidate match on the development sandbox); LLM calls dominate latency.
Portability	SQLite by default; Postgres/Supabase through one environment variable. Embedding backend switchable by one variable.
Testability	Logic is separated from I/O so it can be tested without a database or API key.
Privacy	Resumes are stored locally under random filenames. The prototype has no authentication (see Limitations).

4.3 Software and hardware requirements

Item	Requirement
Operating system	Windows, macOS or Linux
Backend runtime	Python 3.10+ (developed and tested on 3.12); packages: fastapi, uvicorn, sqlalchemy, pydantic, python-dotenv, email-validator, python-multipart, groq, pymupdf, python-docx, numpy, scikit-learn
Frontend runtime	Node.js 18+; React 19, Vite, Tailwind CSS 3
External service	Groq API key (model openai/gpt-oss-120b) and internet access for LLM calls
Optional	sentence-transformers and a one-time model download (about 90 MB) for semantic embeddings
Hardware	Any modern laptop; 4 GB RAM is sufficient (the index is about 16 MB); no GPU needed
Browser	Current Chrome, Edge or Firefox

5. System Architecture

The system has three tiers: a React single-page frontend, a FastAPI backend, and data/AI resources (SQLite database, internship knowledge base with vector index, and the Groq-hosted LLM). Routers handle HTTP concerns only; they load the student profile as a plain dictionary and delegate to the agent or module that owns the logic.

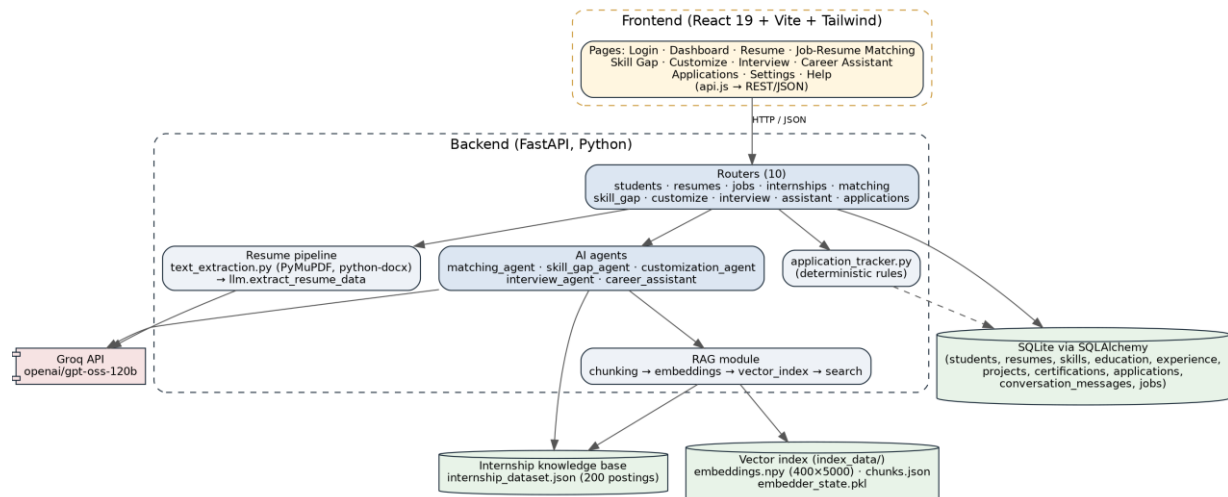


Figure 1. High-level system architecture

5.1 Components

Component	Role
Frontend / interface	Pages for login, resume, matching, skill gap, customization, interview, assistant and applications; one API function per endpoint.
Backend services	Ten FastAPI routers (students, resumes, jobs, internships, matching, skill_gap, customize, interview, assistant, applications).
Student profile management	Student with skills, education, experience, projects and certifications; created by resume parsing.
Resume processing pipeline	Upload, text extraction (PyMuPDF / python-docx), LLM structured extraction, profile persistence.
Internship knowledge base	200 curated postings with required and preferred skills, experience and education requirements.
RAG architecture and vector index	Field-based chunking, TF-IDF embeddings, NumPy matrix index with cosine search.
AI agents	Matching, Skill Gap, Customization, Interview, Career Assistant.
LLM integration	Groq client shared by agents; JSON mode and per-call validation and fallbacks.
Application tracking system	Deterministic rules for status, dashboard, reminders and filtering.

5.2 Data flow

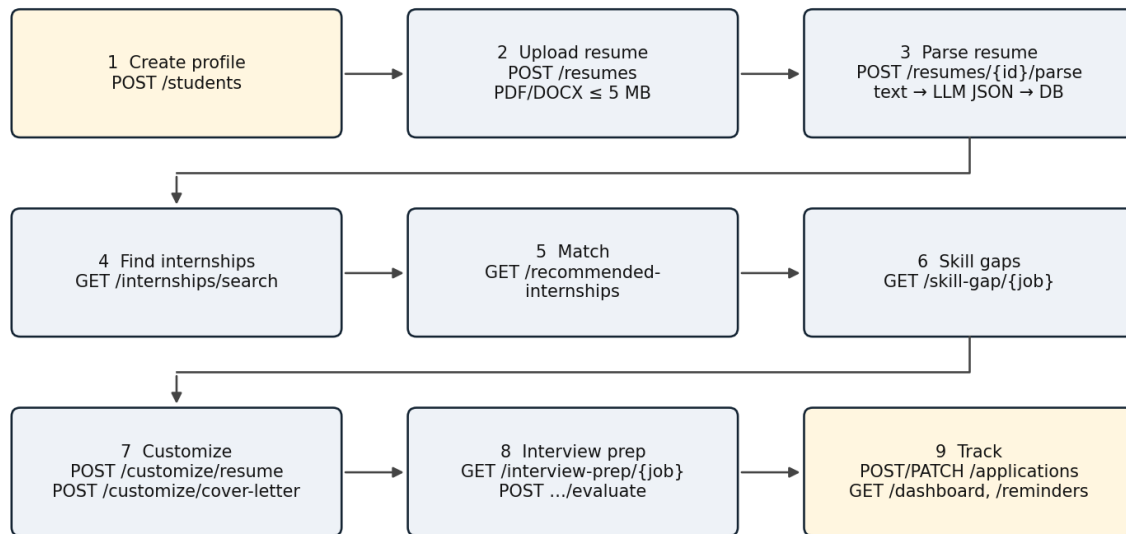


Figure 2. Student workflow and the API call behind each step

5.3 Key design decisions

- **Deterministic core, LLM at the edges.** Scores, gaps, statuses, counts and reminders are plain code, so they are repeatable and testable. The LLM writes explanations and documents and is never trusted for numbers.
- **Retrieve, then score.** Semantic retrieval narrows 200 postings to 15 candidates (recall); an explicit rubric ranks them (precision). Retrieval similarity alone is not the match score.
- **Reuse instead of duplication.** The Interview agent uses the Skill Gap result; the Assistant calls the other agents rather than answering from model knowledge.
- **Swappable infrastructure.** One environment variable changes the database; one changes the embedding backend. The local index mirrors what pgvector would do.

6. Dataset and Knowledge Base

6.1 Overview

Item	Value
Sample job postings	200 (176 distinct titles)
Source	EMSCAD (Employment Scam Aegean Dataset), 17,880 real postings from 2012–2014, University of the Aegean, via a GitHub CSV mirror of the Kaggle “Real / Fake Job Posting Prediction” data
Collection method	Programmatic filtering of the public dataset (no scraping, no live feeds)
Chunks / embeddings	400 chunks (200 overview + 200 requirements); $400 \times 5,000$ TF-IDF matrix
Skill vocabulary	60 terms; 58 distinct skills appear in the final postings

6.2 Job categories

Of the 200 postings, 130 carry a source “function” label: Marketing 34, Engineering 17, Design 11, Information Technology 10, Sales 9, Art/Creative 7, Data Analyst 6, Writing/Editing 5, Consulting 5, Business Development 5, Customer Service 4, and 17 in other functions; 70 are unlabelled. Leading industries are Marketing and Advertising (24), Internet (22), Computer Software (16), Information Technology and Services (14) and Design (9). Locations are mainly US (99), GB (29), GR (18), DE (12) and NZ (8). The most frequent skills are Communication (117 postings), Sales (51), Excel (51), JavaScript (36), Photoshop (33), CSS (31) and HTML (31).

6.3 Preprocessing

Step	Result
Filter genuine postings with “intern” in the title	578 of 17,880
Clean HTML, entities and anonymisation tokens	Plain text descriptions
Extract company name by pattern	96 of 200 identified; 104 “Undisclosed Company”
Extract skills against the vocabulary; split required and preferred by marker words	Median 3 required skills per posting; 23 postings with none
Remove exact duplicates	33 removed
Remove near-duplicates (same company, word-set Jaccard > 0.85)	63 removed
Cap at 6 postings per title and keep the 200 richest	200 postings

Experience level is “Internship” for 116 postings (Associate 8, Entry level 7, Mid-Senior 4, Director 2, unstated 63). Education is Bachelor’s for 43 postings, Some college for 25, Master’s for 8, and unspecified or missing for 116.

7. RAG Pipeline

OFFLINE — build_dataset.py, build_index.py

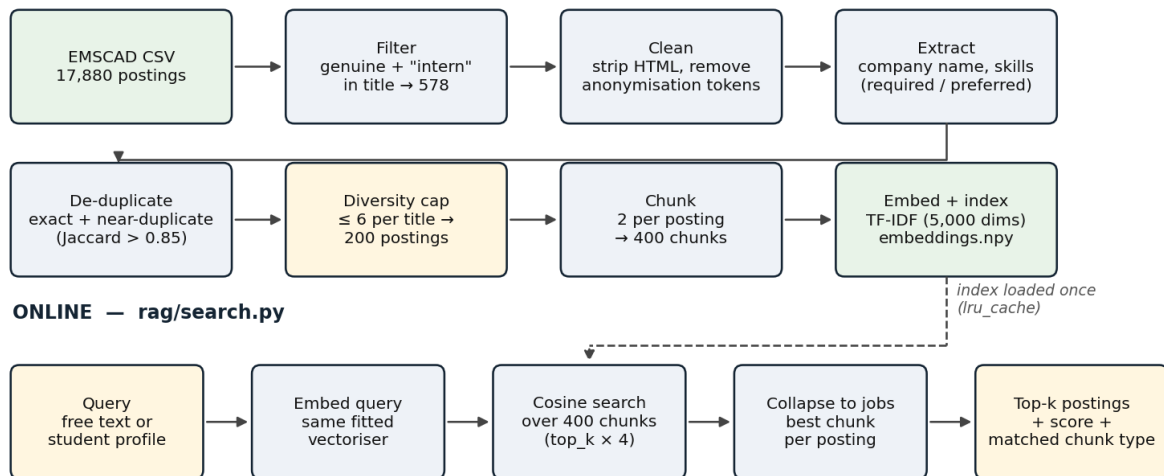


Figure 3. Offline build and online retrieval

7.1 Chunking

Each posting becomes two chunks: an **overview** (title, company, description) and a **requirements** chunk (requirements text, required and preferred skills, experience level, education). Splitting by meaning rather than by character count keeps chunks coherent, so a question about skills and a question about the role retrieve different chunks. A 1,500-character cap protects against outlier descriptions (the median is about 1,260 characters).

7.2 Embedding

The default backend is TF-IDF (English stop-words removed, 5,000 features, unit-normalised) so the system runs offline with no downloads. A sentence-transformers backend (all-MiniLM-L6-v2) is implemented behind the same interface and selected by an environment variable; it has not been run or evaluated, so every retrieval result in this report is TF-IDF.

7.3 Index and search

Embeddings are stored as a NumPy matrix with chunk metadata in JSON. A query is embedded with the same vectoriser, scored against all 400 chunks by dot product (equal to cosine similarity because vectors are unit length), the top four-times-k chunks are kept, and results are collapsed to one entry per posting using its best chunk. The same function serves the search endpoint, the matching agent (query built from target role, skills, experience titles and project titles) and the assistant.

7.4 Retrieval quality

Five hand-written queries from the RAG milestone, run on the TF-IDF index:

Query	Top result	Relevant?
frontend web development internship with React and JavaScript	Business Operations Intern for Social Impact	No

Query	Top result	Relevant?
social media marketing and content creation internship	Social Media & Digital Marketing Internship	Yes
data analysis and Python internship	Engineering Intern (skills include Python)	Yes
graphic design and Photoshop internship	Graphic Design Intern (Paid)	Yes
sales and business development internship	Sales and Marketing Analyst Intern	Yes

Four of five top results were clearly relevant. The miss is instructive: the top posting contains “react” once (as a verb) and “development” as in business development. TF-IDF matches terms with no sense of context, which is the main argument for switching to semantic embeddings. This is why matching retrieves 15 candidates and then re-scores them with the explicit rubric instead of trusting similarity order.

8. Multi-Agent Architecture

Five agents share one student profile and one knowledge base. Four are specialists, each owning one question a student asks; the fifth, the Career Assistant, coordinates them in conversation. The agents are not free-running: each has a defined input, a fixed pipeline and a validated output, which makes behaviour predictable and testable.

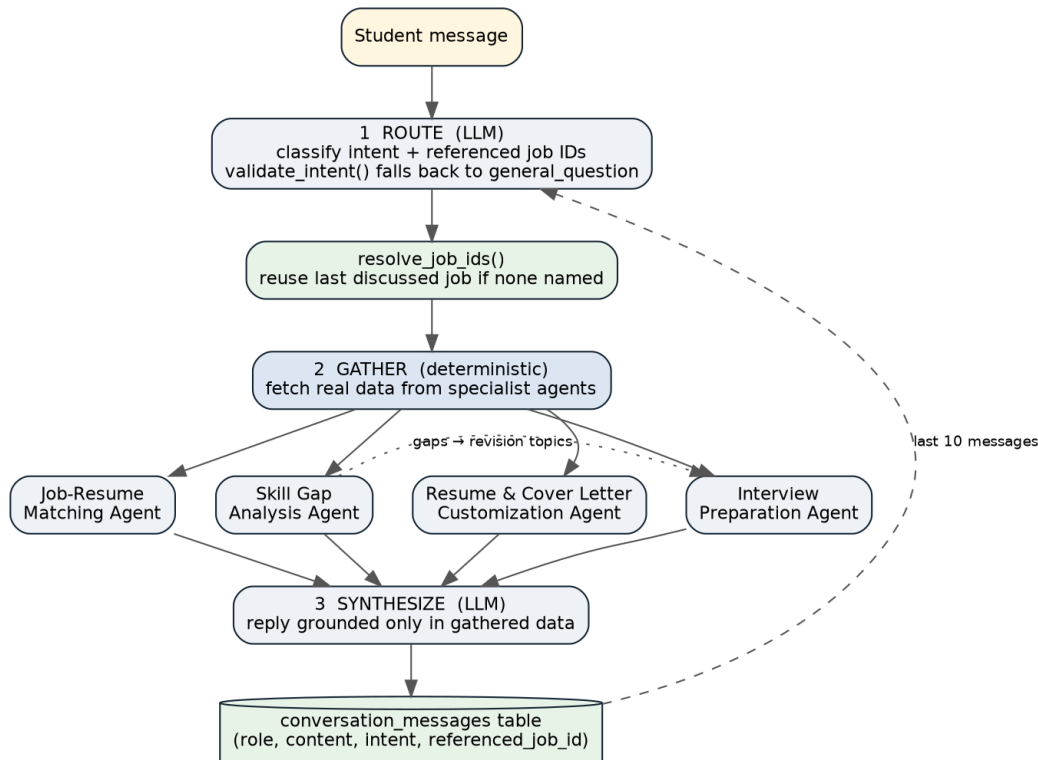


Figure 4. Coordination between the Career Assistant and the specialist agents

Agent	Question it answers	Inputs	Outputs
Job-Resume Matching	Which internships fit me and why?	Profile; knowledge base	Ranked internships, four scores, matched/missing skills, explanation
Skill Gap Analysis	What am I missing for this role and how do I close it?	Profile; one posting	Critical, partial and preferred gaps; experience/education gap; recommendations
Resume & Cover Letter Customization	How do I present myself for this role?	Profile; posting; optional feedback	Tailored resume content or cover letter plus validation flags
Interview Preparation	What will I be asked and what should I revise?	Profile; posting; skill-gap result	Questions in five categories, revision topics, answer feedback
Conversational Career Assistant	Any of the above, in plain language	Message; history; profile	Grounded reply, intent, referenced job IDs

8.1 Coordination pattern

The Assistant uses a route–gather–synthesize loop. **Route**: an LLM call classifies the message into one of seven intents and extracts referenced job IDs; the result is validated and unknown intents become `general_question`. **Gather**: plain code calls the appropriate specialist agents, so the reply is built from real computed data. **Synthesize**: a second LLM call writes a conversational reply using only that data.

Context retention is a mechanism, not a prompt hope: each stored message carries the job it discussed, and when a new message names no job the last discussed one is used.

9. Agent Design and Implementation

9.1 Job-Resume Matching Agent

The agent retrieves 15 candidates by semantic search, scores each, sorts, keeps the top results, and asks the LLM to explain only those.

```
skill    = 0.8 * (required matched / required) + 0.2 * (preferred matched / preferred)
overall = 0.6 * skill + 0.2 * education + 0.2 * experience

education: 100 no requirement / high-school level / met; 60 lower degree held; 40 no
completed degree yet
experience: 100 internship, entry-level or unstated; 80 other; 70 Associate; 20 Mid-Senior,
Director, Executive
```

Skills carry the most weight because they are the most actionable and the most reliably extracted. A student still studying scores 40 on education rather than 0, because most internships are aimed at current students. A real experience mismatch (a Director-level posting) is scored low and shown rather than hidden.

9.2 Skill Gap Analysis Agent

For each required skill the agent checks, in order: exact match (matched); a related skill the student holds from a conservative 16-entry family table such as TypeScript/JavaScript or Django/Flask/FastAPI (partially demonstrated); otherwise critical missing. Preferred skills not held become preferred gaps. Experience and education gaps are reported with a severity. Only then does the LLM explain why each gap matters for this role and suggest a type of project or practice to close it (never a specific paid product). Example on a real posting, “Web Developer (Intern)” requiring CSS, Django, HTML, JavaScript and Python, for a student with JavaScript, React, HTML, CSS, Python, SQL and Git: four skills matched, **Django** critical missing.

9.3 Resume & Cover Letter Customization Agent

Three stages sandwich the LLM. **Rank**: experience, projects and skills are scored by keyword overlap with the posting so the model starts from a prioritised list. **Generate**: prompts state that the profile is the only source of truth and forbid inventing skills, projects, metrics or accomplishments; the student can send free-text feedback (“make it shorter”) to regenerate. **Validate**: a post-check flags skill-like terms (a 26-term closed vocabulary) that appear in the output but not in the profile, returning clean or a list of suspect skills. This is a real check on invented skills, but it cannot detect invented achievements written in prose; that needs semantic fact-checking and is listed as future work.

9.4 Interview Preparation Agent

Revision topics are built directly from the Skill Gap result: critical missing skills are high priority, partial skills medium, and an experience gap adds an experience-expectations topic. The LLM then generates the requested number of questions in each of five categories (Technical, Resume-based, Project-based, Role-specific, HR/general) with preparation guidance. A coverage check confirms all five categories are present. Practice answers are rated strong, adequate or needs_work with strengths and improvements, with the student’s real background supplied so inconsistencies can be noticed.

9.5 Conversational Career Assistant

Seven intents map to deterministic data gathering: recommend (matching agent), explain match and compare (scoring function), explain gap (gap agent), customize (resume or cover letter, chosen by whether the message says “cover letter”), interview prep (one question per category) and general question (semantic search). The last ten messages are loaded for history, and every turn is stored with its intent and referenced job.

9.6 Resume processing and LLM integration

Resume text (first 12,000 characters) is sent to the LLM at temperature 0.1 in JSON mode with rules not to invent content and to keep dates as written. Missing keys default to empty lists. Temperatures are low for extraction, routing and explanation (0.1–0.3), moderate for generation (0.4–0.6). All calls use the Groq model openai/gpt-oss-120b, configurable by environment variable.

10. Application Tracking Module

The module is deliberately free of any LLM: a wrong count on a dashboard is a bug, not a stylistic choice. Its rules live in `application_tracker.py` and are unit-tested independently of the database.

Feature	Behaviour
Status stages	Saved, Planning to apply, Applied, Application under review, Shortlisted, Interview scheduled, Interview completed, Offer received, Rejected, Withdrawn. Anything else returns HTTP 422 listing valid values.
Record	Company, title, description, status, application date, deadline, interview date and status, notes, resume snapshot, cover-letter snapshot.
Dashboard (6 metrics)	Total, active (not Offer/Rejected/Withdrawn), upcoming deadlines (next 7 days, non-terminal), interviews scheduled, offers received, rejected.
Reminders (4 types)	Upcoming deadlines; upcoming interviews; follow-ups suggested (Applied for 10 days or more); pending applications (Saved or Planning to apply). Window configurable from 1 to 90 days.
Filtering	Company and title (case-insensitive substring), status, deadline before / after.
Operations	Create, list, read, update (partial), delete; all scoped to the owning student.

11. Testing and Evaluation

11.1 Strategy

Level	What it checks	How
Unit tests (10)	Score monotonicity, gap classification with skill families, revision-topic priorities, question-coverage check, hallucination check, intent validation and context fallback, status validation, dashboard counts, reminders, filtering	pytest on the deterministic functions; no database or key needed
End-to-end test (10)	Profile → upload → parse → search → match → skill gap → customize → interview → assistant history → application CRUD/dashboard; duplicate email (409), wrong file type (415), oversize file (413), unknown job (404), invalid status (422)	FastAPI TestClient on the real app, real SQLite DB and real RAG index; Groq client replaced by a deterministic fake
Retrieval evaluation	Precision@5 of recommendations for five different student profiles	evaluation/run_eval.py
Score consistency	Overall score never decreases as skill overlap increases	Controlled test varying overlap from 0/10 to 10/10
Extraction spot check	Required/preferred skills against raw posting text	Manual review of three postings

Result of the automated suite: **19 passed, 1 expected failure** (a deliberately recorded known limitation, see Section 13). The fake LLM verifies wiring, validation and persistence; it says nothing about the quality of real model text.

11.2 Retrieval evaluation (precision@5)

Profile	Naive keyword check	Final check (title, industry, function and skills)
Frontend developer	100%	100%
Marketing	100%	100%
Design	100%	100%
Data science	60%	80%
Sales / business	60%	100%
Average	84%	96%

The jump from 84% to 96% is an improvement in the evaluation, not the agent: generically titled postings such as “BI Interns” (business intelligence) and “Growth Wunderkind” (asking for SQL, Data Analysis, Google Analytics) are genuine matches that a title-only check missed. The one remaining miss, “BI Interns” for the data-science profile, was matched correctly through the student’s Tableau skill (a preferred skill of the posting, score 92); the evaluation keyword list simply lacked “tableau”. Counting it as relevant gives 100% effective precision, but the raw 96% is reported as the measured figure. I re-ran the harness for this report and reproduced the stored results exactly.

11.3 Score consistency

Required skills matched (of 10)	0	1	2	3	4	5	6	7	8	9	10
Overall score	40	45	50	54	59	64	69	74	78	83	88

Strictly increasing, so more overlap never lowers a score. The floor of 40 and ceiling of 88 occur because the test posting states no education or experience requirement (each then contributes its full 20% share, 40 points in total) and has no preferred skills (so the skill score tops out at 80, contributing 48).

11.4 What was not evaluated

- **Quality of LLM output** (resume extraction accuracy, explanations, tailored documents, interview questions, chat replies). It needs a live API key and human review; the demonstration checklist in Appendix A is the protocol.
- The sentence-transformers embedding backend.
- Frontend automated tests, load and concurrency testing, and security testing.

12. Results and Analysis

12.1 Worked example

A sample student (JavaScript, React, HTML, CSS, Python, SQL, Git; Bachelor's degree in Computer Science; target role Frontend Intern) was run through retrieval and scoring on the 200-posting knowledge base. The five recommendations:

Overall	Internship	Skill / Edu / Exp	Matched required	Missing required
100	Web Design Intern Part Time, Paid	100 / 100 / 100	— (skills are all preferred: CSS, HTML, JavaScript)	—
88	Graduate Intern – Python and Mongo DB	80 / 100 / 100	css, javascript, python	—
88	Software Engineering Intern	80 / 100 / 100	html, javascript, sql	—
88	Internship – Software Engineering (Summer)	80 / 100 / 100	html, javascript, sql	—
72	Junior WebDeveloper [Intern]	53 / 100 / 100	css, html, javascript, sql	php, sales

All five are web or software roles that use the student's actual skills, and the one with gaps (PHP, Sales) shows them. Retrieval plus scoring for this request took about 1.2 seconds on the development sandbox (cold start, single run, no LLM calls).

12.2 Analysis

- **Retrieve-then-score works.** Retrieval is noisy (one of five queries missed), but the rubric re-ranking over 15 candidates produced relevant top-5 lists for all five evaluation profiles (96% measured).
- **Scores are explainable and monotonic.** Each recommendation shows which skills matched and which are missing, and more overlap never lowers a score.
- **Grounding checks are cheap and effective for skills.** The coverage and unsupported-skill checks convert prompt rules into verifiable post-conditions, within the limits noted.
- **Two scoring behaviours deserve attention.** Postings with no required skills (23 of 200) start from a skill score of 80, so a posting whose skills are all “preferred” can reach 100/100 as in the first row above. And degree matching keys on the literal “bachelor’s”, so “Bachelor of Engineering” or “B.E.” scores 40 instead of 100 against a Bachelor’s requirement. Both are documented, the second has an expected-failure test, and both are straightforward to fix.
- **The weakest component is TF-IDF retrieval.** It underlies every recommendation. Semantic embeddings are the single highest-value improvement.

13. Limitations

Area	Limitation
Dataset	200 postings from 2012–2014 mostly in the US and UK; stack mix skews to that era (little cloud or AI). 104 company names are undisclosed; 23 postings list no required skills. Skills come from a 60-term vocabulary, so unlisted skills are invisible.
Retrieval	TF-IDF has no word-sense or paraphrase awareness. Semantic backend implemented but not evaluated.
Matching	Exact-string skill comparison (no JS/JavaScript synonyms); crude degree matching; zero-required-skill postings inflate scores; unpunctuated bullet lists can push required skills into the preferred bucket.
Gap analysis	Skill-family table has only 16 entries.
Customization	Hallucination check covers a 26-term skill vocabulary, not invented achievements or exaggeration.
Profile	Re-parsing overwrites the profile; no manual editing or merging; no confidence score for extraction; scanned PDFs are not supported (no OCR).
Security and privacy	No authentication (email-only login); file type trusted from the upload header; resumes stored unencrypted on local disk; the repository currently contains 39 uploaded resume files.
Frontend	Dashboard and Settings are static placeholders; the “Apply” button on matches is inert; customised documents are not saved to an application from the UI; no routing library in use.
Evaluation	LLM output quality, sentence-transformers retrieval and real users were not evaluated; precision@5 uses five synthetic profiles and an automatic relevance proxy.
Scale	Brute-force search and SQLite are suitable for hundreds of chunks and a single user, not for production traffic.

14. Future Scope

Priority	Improvement
High	Switch to sentence-transformers embeddings, rebuild the index and re-run the precision@5 evaluation to quantify the gain.
High	Add real authentication and per-user authorisation; validate file signatures; stop committing uploads; add retention controls.
High	Wire the Dashboard to live data; make “Apply” create a tracked application and attach the tailored resume and cover letter as snapshots.
Medium	Fix matching edge cases: normalise degree names (B.E., B.Tech, Bachelor of ...), neutral skill score when no required skills exist, skill synonym and alias normalisation.
Medium	Refresh and widen the knowledge base with current postings from APIs or feeds, including Indian job markets; scheduled re-indexing.
Medium	Add manual profile editing, confidence indicators for extracted fields and OCR for scanned resumes.
Medium	Semantic fact-checking of generated documents against the source resume to catch invented achievements.
Low	pgvector or FAISS for larger corpora; Postgres with migrations; deployment with environment-based CORS.
Low	Email or calendar reminders; resume export to PDF/DOCX; mock interviews with voice; user study to measure outcomes.

15. Conclusion

AI Career Companion shows that a RAG-based, multi-agent system can support a student from resume to tracked application in one coherent workflow. Its main strength is a design that keeps the exact parts exact: scores, gaps, statuses, dashboard counts and reminders are deterministic and tested, while the language model explains, drafts and converses within validated boundaries. Generated documents are checked for unsupported skills, interview questions are checked for category coverage, and the assistant answers from real agent output rather than from free-floating model knowledge.

The measured results are encouraging but honest about their limits: 96% precision@5 on five profiles, monotonic scoring, and a passing automated suite of 19 tests with one documented known-limitation. The main weaknesses are the TF-IDF retrieval baseline, a small and dated dataset, a few scoring edge cases, an unauthenticated prototype with some placeholder pages, and the absence of an evaluation of live LLM output. Each is documented with a concrete next step in Section 14, and none requires changing the architecture. The system is ready to demonstrate end to end (Appendix A) and to extend.

References

- Vidros, S., Kolias, C., Kambourakis, G., Akoglu, L. (2017). Automatic Detection of Online Recruitment Frauds: Characteristics, Methods, and a Public Dataset. *Future Internet*, 9(1), 6. (EMSCAD dataset)
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS* 33.
- Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP-IJCNLP*.
- FastAPI, SQLAlchemy, React, Vite, scikit-learn and Groq API documentation.